A3-PS023-TC246

Memory-Aware VM Scheduling with Atomic Resource Reservation on KVM Hosts

Rajya vardhan .M
Shivani S R
rajyavardhan200@gmail.com
shivani.s.raravi@gmail.com

Problem Description

A cloud hypervisor needs to place VMs on hosts without running out of memory.

Memory-blind placement: a scheduler that ignores free RAM can pick a host that cannot actually fit the VM, so the launch fails.

Double booking: two requests that arrive together can read the same free-memory value and both claim it.

State drift: a host crash or a manual change leaves the records out of step with what is really running.

No visibility: users cannot tell why a request failed or what state their VM is in.

Goal
Build a controller and per-host worker agents on KVM that place VMs by available memory, reserve that memory atomically, and keep the records in line with the real VM state.

Background

Why VM placement is a memory problem

KVM turns a Linux host into a hypervisor [1]. Every VM is given a fixed amount of RAM taken from its host, so choosing a host is a bin-packing decision across the cluster. If a host is asked to hold more than it has, memory has to be reclaimed or swapped, and VM performance suffers [2].

Large cluster schedulers handle this with a central scheduler and per-machine agents, and they have to deal with many requests competing for the same resources [3][4]. This design applies the same split to a small cloud of two KVM hosts.

Hosts
Host 1 and Host 2 run KVM + Linux. Each reports its memory and health.

VM request
A request states the memory the VM needs. It is checked before any host is touched.

Main risk
Overcommit and double booking when memory is not reserved safely.

Literature & RFC Review

Only the studies that shape this design are listed.

Work	What it contributes	How we use it
KVM [1]	Linux kernel-based hypervisor	Runs the VMs on Host 1 and Host 2
VMware ESX [2]	Memory management for VMs on one host	Shows why memory must be tracked per VM
Borg [3]	Central controller with per-machine agents	Same controller and worker split
Omega [4]	Shared state with optimistic concurrency	Motivates atomic reservation of resources
Xen [5]	Early open hypervisor with resource isolation	Background for the hypervisor API client

Proposed Solution

One central controller plans. One worker agent per host acts.

Controller (centralized)

- API Layer: receives and validates requests
- Scheduler: memory-aware host choice
- Reservation Manager: atomic reserve / release
- State Reconciler & Health Monitor

Worker Agent (per host)

- Task Executor: runs create / start / stop
- Hypervisor API Client: local API to KVM
- Host Monitor: reports memory and health

Request path: validate, pick a host with enough memory, reserve it, send the task over an authenticated link, run it on KVM, then reconcile the result. A database holds VMs, hosts, reservations and logs.

Optimization Proposed

Each change replaces a weak point of a basic VM launcher.

Area	Before	After
Placement	Host picked without checking memory	Scheduler reads available memory and picks a host that fits
Concurrency	Check, then act: two requests can take the same RAM	Atomic reserve, commit and release
Host health	No view of host memory	Workers send memory and health reports
State drift	Records can differ from real VMs	Reconciler compares actual status and cleans up
Security	Open channel to workers	Authenticated task channel
Visibility	No request history	Requests and VM state logged in the database

Solution Architecture & Design

Visual: the controller, two worker agents, two KVM hosts and the database.

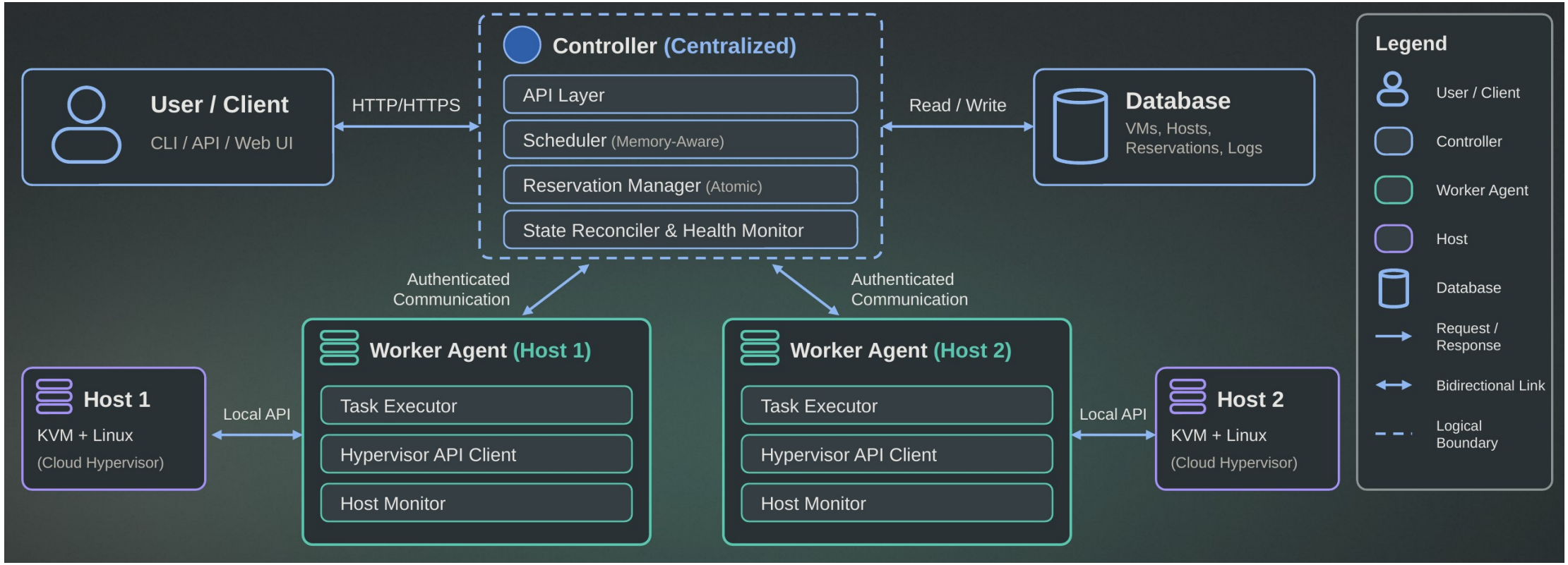


Figure: System Architecture UML Diagram

Data Flow Diagram

How a request moves from the client to a KVM host and back.

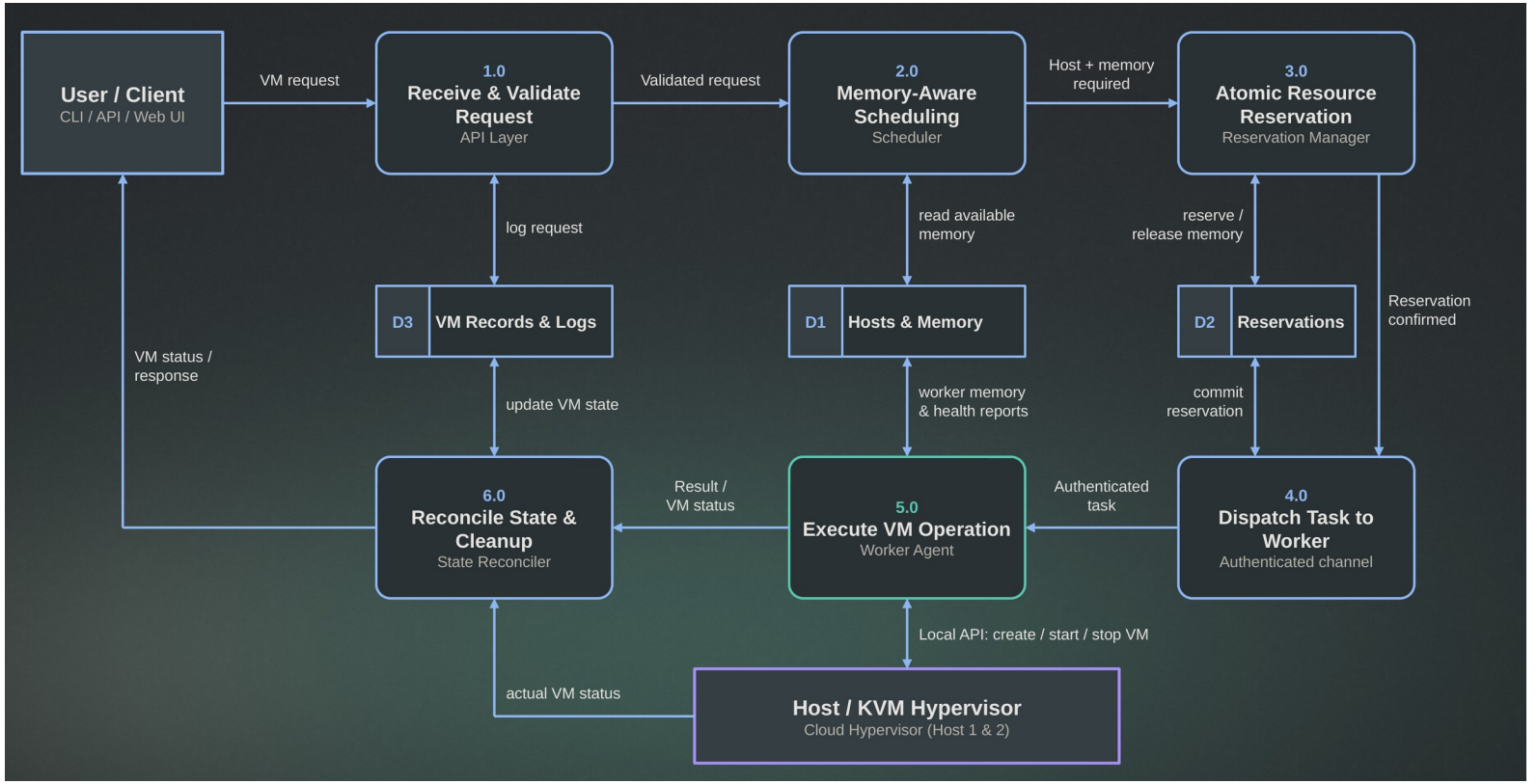


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Five phases, each tested on both KVM hosts before the next.

Phase 1	Phase 2	Phase 3	Phase 4	Phase 5
Controller, API layer and database	Worker agents and KVM API client	Memory-aware scheduler and atomic reservation	State reconciler and health monitor	End-to-end test and demo

Method: build in small steps, test each one against both hosts, and keep the controller and workers on the authenticated channel from the first phase.

Bibliography I

[1] A. Kivity et al. “kvm: the Linux Virtual Machine Monitor.” In: *Proceedings of the Linux Symposium* vol. 1 (2007), pp. 225–230.

[2] C. A. Waldspurger. “Memory Resource Management in VMware ESX Server.” In: *OSDI* (2002), pp. 181–194.

[3] A. Verma et al. “Large-scale cluster management at Google with Borg.” In: *EuroSys* (2015).

[4] M. Schwarzkopf et al. “Omega: flexible, scalable schedulers for large compute clusters.” In: *EuroSys* (2013), pp. 351–364.

[5] P. Barham et al. “Xen and the Art of Virtualization.” In: *SOSP* (2003), pp. 164–177.

Thank You for Your Attention!